

OS LAB 10 TASKS

Syed Sultan Ahmed, 24K-0585

BCS-4H

1)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define READERS 5
#define WRITERS 2

int readCount = 0;
sem_t mutex, wrt, queue;

void *reader(void* arg) {
    int id = *(int*)arg;
    while(1) {
        sem_wait(&queue);           // join
        sem_wait(&mutex);
        readCount++;
        if(readCount == 1)
            sem_wait(&wrt);         // first reader locks
        sem_post(&mutex);
        sem_post(&queue);          // release

        printf("Reader %d is reading\n", id);
        sleep(1);

        sem_wait(&mutex);
        readCount--;
        if(readCount == 0)
            sem_post(&wrt);         // last reader unlocks writers
        sem_post(&mutex);
    }
}
```

```

        sleep(1);
    }
}

void *writer(void* arg) {
    int id = *(int*)arg;
    while(1) {
        sem_wait(&queue);          // join
        sem_wait(&wrt);
        sem_post(&queue);          // release

        printf("Writer %d is writing\n", id);
        sleep(1);

        sem_post(&wrt);
        sleep(1);
    }
}

int main(int argc, char* argv[]) {
    int id[READERS];
    pthread_t r[READERS];
    pthread_t w[WRITERS];

    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);
    sem_init(&queue, 0, 1);

    for(int i = 0; i < READERS; i++) {
        id[i] = i+1;
        pthread_create(&r[i], NULL, reader, &id[i]);
    }
    for(int i = 0; i < WRITERS; i++) {
        id[i] = i+1;
        pthread_create(&w[i], NULL, writer, &id[i]);
    }
    for(int i = 0; i < READERS; i++)
        pthread_join(r[i], NULL);
    for(int i = 0; i < WRITERS; i++)

```

```

        pthread_join(w[i], NULL);

    sem_destroy(&mutex);
    sem_destroy(&wrt);
    sem_destroy(&queue);
}

```

```

sultan@sultan-VirtualBox:~/Downloads/Labs/OS/Lab10$ gcc 4_reader-writ
Reader 1 is reading
Reader 3 is reading
Reader 4 is reading
Writer 1 is writing
Reader 5 is reading
Reader 1 is reading
Writer 2 is writing
Reader 4 is reading

```

2)

```

#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>

#define NUMPRODUCER 3
#define NUMCONSUMER 3
#define N 5

sem_t empty, full, mutex;
int buffer[N] = {0};
int in = 0, out = 0;
int orderID = 1;

void* producer(void* args) {
    int* id = (int*)args;
    while(1) {
        int order = orderID++;

        sem_wait(&empty);

```

```

        sem_wait(&mutex);

        buffer[in] = order;
        printf("Customer  %d placed  Order #%d at slot [%d]\n", *id, order,
in);

        in = (in + 1) % N;

        sem_post(&mutex);
        sem_post(&full);

        sleep(rand() % 2);
    }
    free(id);
}

void* consumer(void* args) {
    int* id = (int*)args;
    while(1) {
        sem_wait(&full);
        sem_wait(&mutex);

        int order = buffer[out];
        printf("Agent      %d picked  Order #%d from slot [%d]\n", *id,
order, out);
        out = (out + 1) % N;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(rand() % 3);
    }
    free(id);
}

int main(int argc, int* argv[]) {
    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_t producers[NUMPRODUCER];

```

```

pthread_t consumers[NUMCONSUMER];

for(int j = 0; j < NUMCONSUMER; j++) {
    int* id = (int*)malloc(sizeof(int));
    *id = j+1;
    pthread_create(&consumers[j], NULL, consumer, id);
}
for(int j = 0; j < NUMPRODUCER; j++) {
    int* id = (int*)malloc(sizeof(int));
    *id = j+1;
    pthread_create(&producers[j], NULL, producer, id);
}
for(int j = 0; j < NUMCONSUMER; j++)
    pthread_join(consumers[j], NULL);
for(int j = 0; j < NUMPRODUCER; j++)
    pthread_join(producers[j], NULL);

sem_destroy(&full);  sem_destroy(&empty);  sem_destroy(&mutex);
}

```

```

sultan@sultan-VirtualBox:~/Downloads/Labs/OS/Lab10$ gcc 2_producer-co
Customer 1 placed Order #1 at slot [0]
Customer 2 placed Order #2 at slot [1]
Agent 1 picked Order #1 from slot [0]
Customer 3 placed Order #3 at slot [2]
Agent 3 picked Order #2 from slot [1]
Agent 2 picked Order #3 from slot [2]
Customer 3 placed Order #4 at slot [3]
Customer 2 placed Order #5 at slot [4]

```